

Design Statement — Simulating the Amazon Homepage Layout (HTML + CSS)

1. Basic Information

This document is the design statement for the assignment “Simulating the Amazon Homepage Layout (HTML + CSS)”. The submitted archive contains index.html, the css folder and the img folder, all built with plain HTML and an external CSS file.

Student ID	25532012
Student Name	SHI Hau Yan
Personal GitHub link	https://github.com/JC0526/amazon.git

2. Design Rationale and Overall Approach

2.1 Page Block Division

The Amazon homepage was divided into five main blocks from top to bottom. The basis of the division is the visual strip structure of the real page: every strip has a distinct background colour and a distinct function, so the boundaries between strips are clearly visible when the page is rendered. The five blocks are listed below.

Block 1, the top navigation bar, is a full-width dark strip containing the logo placeholder, the delivery-address link, the search bar and the account, orders and cart links. Block 2, the secondary navigation bar, is a full-width dark strip with the category links and the English-shop link. Block 3, the main content area, holds Row A (five colour category tiles), Rows B to G (six rows of four white cards, each card containing a 2 by 2 thumbnail layout) and the sign-in banner. Block 4 is the back-to-top strip. Block 5 is the footer, which contains four link columns, a language / currency / region bar, a four-row service-links matrix and the legal links with the copyright line.

Block	Main content	Vertical arrangement	Horizontal arrangement
1. Top navigation bar	Logo placeholder, delivery address, search bar, account links and cart	Normal flow, full-width strip	Groups floated left and right
2. Secondary navigation bar	Category links and English-shop link	Normal flow, full-width strip	Links floated left, English link floated right
3. Main content area	Row A (5 tiles), Rows B–G (6 × 4 cards), sign-in banner	Rows stacked in normal flow	Tiles and cards floated per row; thumbnails floated

Block	Main content	Vertical arrangement	Horizontal arrangement
			inside cards
4. Back-to-top strip	Centred text band	Normal flow	—
5. Footer	4 link columns, region bar, services matrix, legal links	Blocks stacked in normal flow	Columns, options and service items floated

2.2 Box Model Control of Sizes and Spacing

Each block's size and spacing were controlled with width, height, border, padding and margin. The fixed-width blocks use a 1400px container (the top navigation inner, the secondary navigation inner and the shared .container class) and are centred horizontally with margin: 0 auto. The footer link columns use a narrower 964px container and the service-links matrix uses 1000px, matching the proportions of the real page.

The search bar is the part where the box model had to be calculated most carefully. The page uses the default content-box sizing, so the rendered width of an element is width plus padding plus border. The category label is 100px + 10px padding-left + 1px border-right = 111px, and the input is 589px + 20px horizontal padding = 609px. Because 111 + 609 = 720px, the two floated children exactly fill the 720px search bar without overflow or a leftover gap; this calculation is recorded in a CSS comment above the input rule.

The product cards use percentage widths: width: 24% with margin-right: 1.333% as the gap, and the last card in every row uses a .last class that sets margin-right to 0. Four cards therefore fill one row without wrapping. The five colour tiles follow the same pattern with width: 19% and margin-right: 1.25%. The sign-in banner is a fixed-width block (width: 820px) centred with margin: 10px auto 20px. Heights are set where a fixed strip needs a controlled size; for example, the search bar is 40px high and the label text is centred vertically with line-height: 40px.

2.3 Normal Flow versus Float

The first principle of webpage layout was followed throughout: for vertical arrangement use normal flow, and for horizontal arrangement use float. Normal flow is used for the whole page body, <main>, the back-to-top strip and the footer, because these are simply strips stacked one after another. Float is used for every line of content that must sit side by side: in the top navigation, the delivery address and the search bar are floated left and the account group is floated right; in the product rows the tiles and cards are floated left; inside every card the four thumbnails are floated left two per row; and in the footer the four columns, the three

option boxes and the service-links rows are all floated. The reason is that normal flow only stacks elements vertically; when two or more elements have to share one line, float is the layout method that takes them out of the flow and places them side by side.

2.4 Handling the Parent-Height Collapse

Floating an element takes it out of the normal flow, so the parent container's height collapses to zero and the following blocks move upwards. This collapse appeared in almost every floated container, and three different clearing methods were used in different places.

Method 1, the extra-tag method, is used in the secondary navigation bar: a `<div class="clear"></div>` with `clear: both` is placed at the end of the floated links. This position was chosen because the navigation has a simple one-line structure, so an extra tag is easy to see and to maintain. Method 2, `overflow: hidden`, is applied to `.search`, `.row5`, `.row4`, `.card-thumbs`, `.footer-cols`, `.container`, `.global-inner` and `.fs-row`. It needs no extra tag and no extra class, which makes it the cleanest choice for containers whose content does not need to overflow the box; on the search bar it also clips the rounded corners. Method 3, the double pseudo-element `.clearfix`, is applied to `.nav-inner` through the reusable `.clearfix::after` class. Because the class can be reused on any floated parent without polluting the HTML, it is the most maintainable choice for the top navigation.

3. Adjustment Approach and Plan

The layout went through several iterations. The main problems I met, how I located them, and how I fixed them are recorded in Table (see also the screenshots in Section 5).

Version / problem	What happened	How I located and fixed it
Search input disappeared	After the change the input box vanished; only the grey label was visible.	Root cause: the label's real width is 111px (100 + 10 padding-left + 1 border-right), not 110px; $111 + 610 = 721\text{px} > 720\text{px}$, so the input wrapped to the next line and was clipped by the bar's <code>overflow: hidden</code> . I set the input to 589px (609px with padding): $111 + 609 = 720\text{px}$.

4. Reflection

Writing this page with float combined with the box model was noticeably more awkward than I expected. The most troublesome parts were vertical centring, manual percentage arithmetic and the clearing recipe. With float there is no direct way to centre content vertically, so a fixed-height bar has to be centred with line-height or with the inline-block trick, and a

card title has to be pushed with padding. Every gap between floated items has to be calculated by hand as a percentage, and the .last modifier has to be remembered for every row. Clearing floats also feels like an experience-based formula rather than something with an obvious logic: three different methods exist, and choosing between them is not always clear.

If I were to learn a newer layout method later, such as flexbox, I would want it to solve exactly these three points: horizontal and vertical centring without calculation, automatic distribution of the gaps between items, and no need for clearing recipes. I would also want it to keep the simple mental model of normal flow for vertical stacking, which was the part of this assignment that felt natural.

5. Screenshots and Supporting Evidence

Figures 1 to 5 show the complete final page rendered in the browser, captured in five segments from top to bottom. Figure 6 shows the public GitHub repository page.

For the before / after comparison, Figure 7 shows the disappeared input (development evidence, the bug), and Figure 8 shows the fixed search bar.

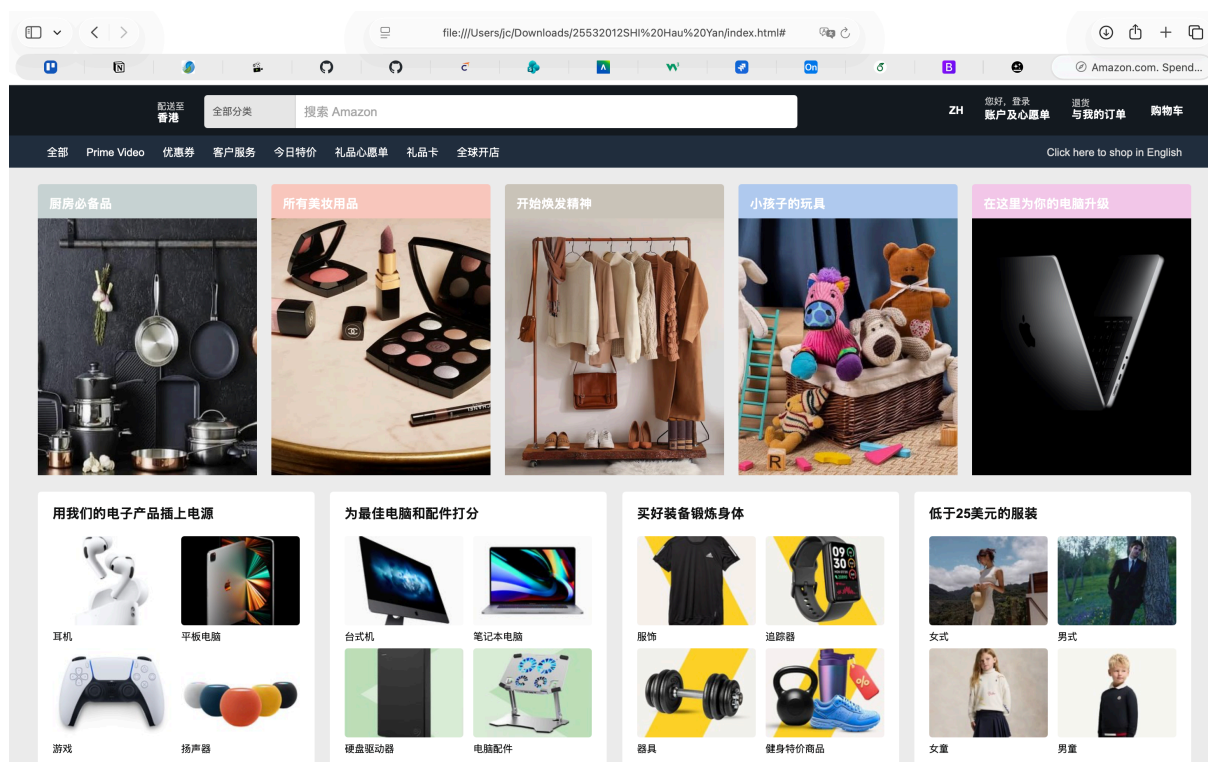
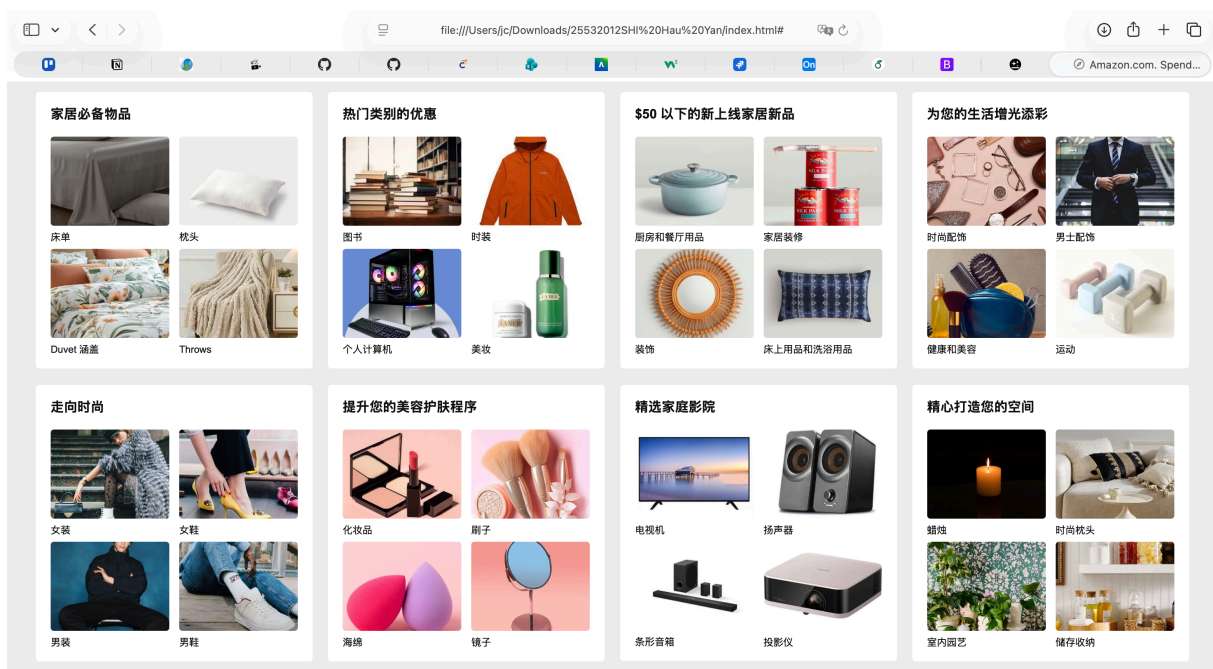
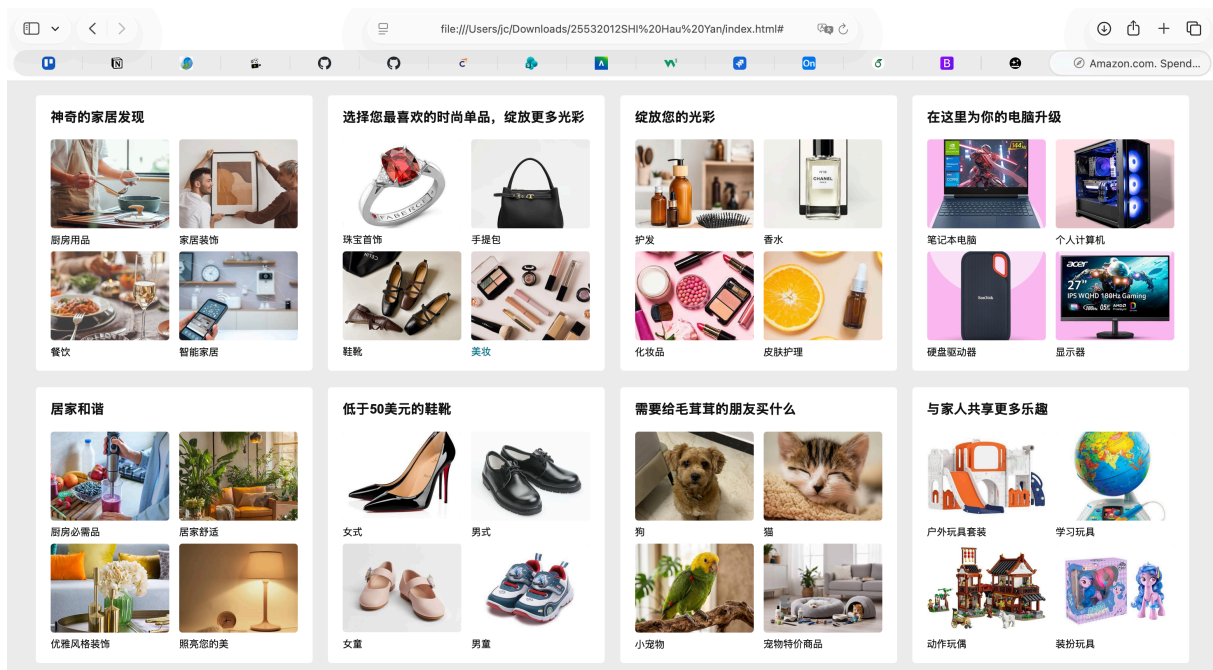


Figure 1. Top navigation bar and product rows A to B



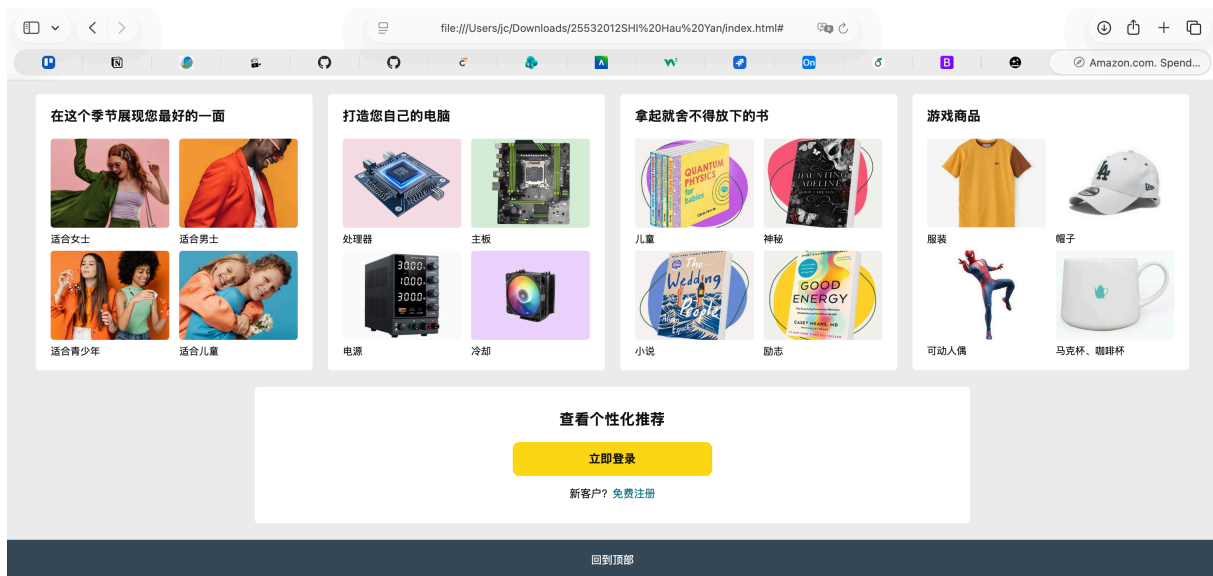


Figure 4. Product row G and the sign-in banner



Figure 5. Footer blocks and the copyright notice

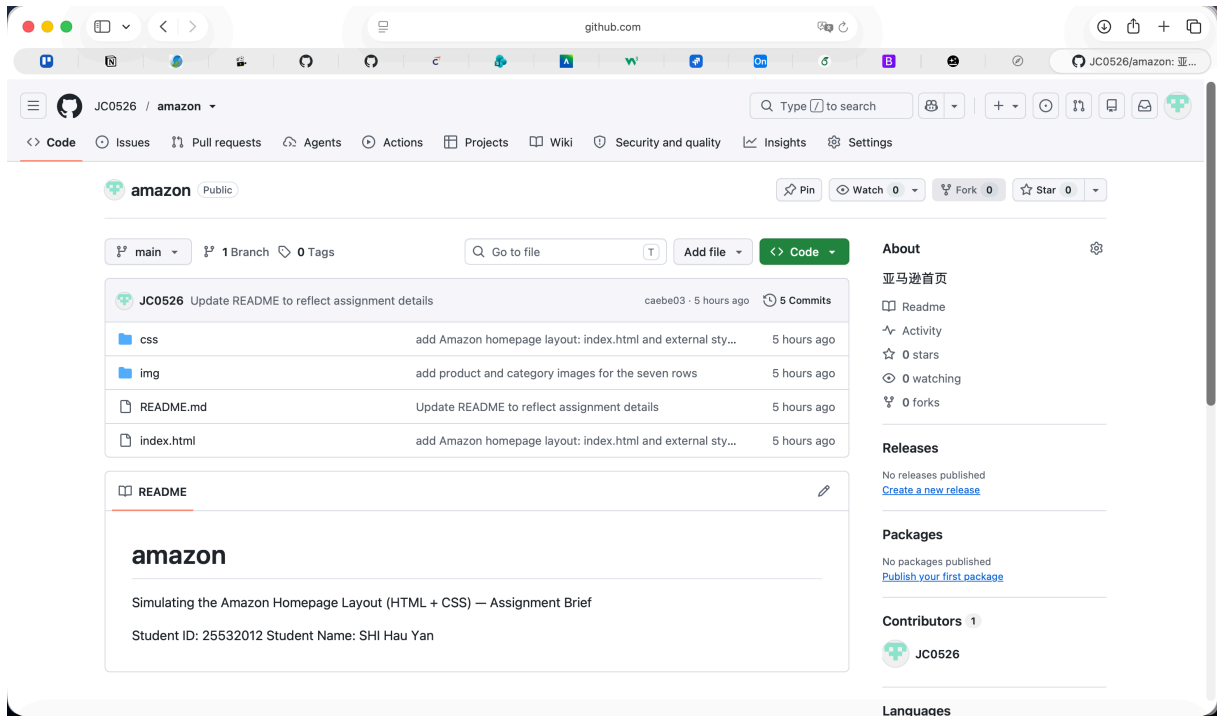


Figure 6. The public GitHub repository JC0526/amazon



Figure 7. Development evidence: the search input disappeared (bug)

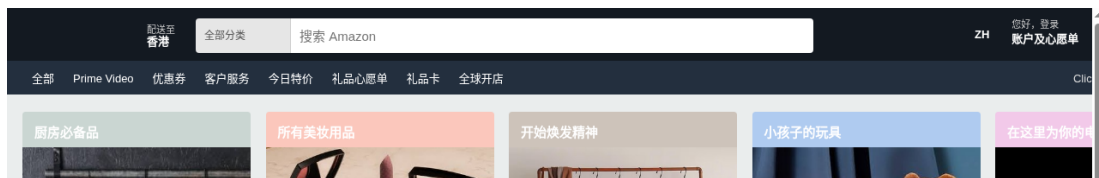


Figure 8. The search bar after the fix: grey label + white input, 720px, no clipping

6. AI Usage Statement

I used AI assistance (Doubao) during this assignment, and I state here honestly at which stage, what I asked, what the AI gave me, and what I changed myself.

- I asked the AI to explain float clearing methods and to help find why the search input disappeared. The AI pointed out that the category label's rendered width was 111px (100 + 10 padding + 1 border) and that $111 + 610 = 721\text{px}$ overflowed the 720px bar, so the input wrapped and was clipped by overflow: hidden.
- I asked the AI to teach me the code sequence for web pages, then referred to the code outline provided by the AI and finally wrote the code myself.
- I asked the AI to check the code against the assignment restrictions. It reported the areas that needed comments and verified the structure.

- This design statement document had its outline generated with AI assistance and then reviewed and written by me.

7. GitHub Commit History

The repository <https://github.com/JC0526/amazon.git> is public and was created on 22 September 2026 with an initial commit that contained only a README file. The complete source code (index.html plus the css and img folders) has since been added in staged commits: “add Amazon homepage layout: index.html and external stylesheet”, “add product and category images for the seven rows”, “update README with assignment information” and a later “Update README to reflect assignment details”. Each commit has its own descriptive message, so the commit history shows that the work was completed in stages rather than pushed in one bulk submission. The repository now contains 5 commits in total, as shown in Figure 6.